ISTA130 Fall 2021 Final Review

General Questions:

1) How much time will we have to complete the final? Will we have less since it's open-note?

> You will have 2 hours, just like any other final exam at the University! That does not change because it's open-note. If you are a DRC student, you will also have your time accommodations applied (any other accommodations should be able to be made by you, too, since we aren't dealing with a proctored system).

2) Will the final cover the topics from the entire semester or focus more on post-midterm topics?

> The final will cover topics from the entire semester, since post-midterm topics build on pre-midterm topics. There's no way for it not to be cumulative!

3) How many questions will the final have? How many coding questions?

> The final contains 47 questions, with a similar distribution to the midterm. 44 of those questions are multiple choice (like the quizzes!) and 3 of them are function writing drills. The distribution of points is the same as the midterm as well - 70% for multiple choice, 30% for the function writing drills

4) Will you grade harder since it's open-note?

> We won't grade explicitly "harder", but we will probably make fewer exceptions to the grading rubric. As always, we will do our best to grade fairly, and since you have notes and your command line you should be able to test the code you write (and even test your multiple choice answers).

5) Will the final review session be recorded?

> Yes! The final review session will be recorded.

When traversing a file is it always good practice to use strip() at the start?

> Generally, it depends on how you want to traverse the file. Sometimes it's necessary to have whitespace included at the ends of the string for what you want to do. However, for our purposes thus far, it's been good practice (and continues to be so for the final).

I want to understand the process of opening and writing a file in python again. And how to use open files through loops.

The different methods we can open csv files and how to work with them in functions

To open a file in "read mode": `fp1 = open(filename, "r") // open(filename)`
To open a file in "write mode": `fp2 = open(filename, "w")`
To read a file line-by-line:

```
for line in fp1:
    ...do something... (probably with 'line')
```

To write to a file: `fp2.write("a string containing some things\n")`
After you're done working with a file (this is particularly important when writing):

```
fp1.close()
fp2.close()
```

Let's do an example to answer both of these questions:

Write a function called `csv_sum` that takes a filename and creates a new file called "<filename>_summed.csv" containing the sums of every line of numbers in the original csv file. The numbers are in a CSV (comma separated value) format. For instance, if the contents of a file are:

```
12,3,2
-5
10,20,-10,8.3
```

The new file should contain:

```
17.0
-5.0
28.3
```

(see next page)

```
def csv_sum(filename):
    '''
    Documentation exists here
    '''
    infile = open(filename) # , "r")
    new_fname = filename[:-4] + "_summed.csv"
    outfile = open(new_fname, "w")

    for line in infile:
        line = line.strip().split(',') # line is a list of strings
        line_sum = 0
        for el in line: # or "for i in range(len(line))" (w/ indexing)
            line_sum += float(el) # or "= line_sum + float(el)"
        outfile.write(str(line_sum) + "\n")
    infile.close()
    outfile.close()

    # consider another version, what if we wanted to create
    # a list of values or a dictionary?
```

Watch the review for other versions!

Applying indexing and slicing to lists

Given we have this list:

```
doggos = ['Freya', 'Harper', 'Rocket', 'Roscoe']
```

What is the output of:

1. `doggos[2] => "Rocket"`
2. `doggos[-1] => "Roscoe"`
3. `doggos[:3] => ['Freya', 'Harper', 'Rocket']`
4. `doggos[0][2] => "e"`
5. `doggos[1] * 2 => "HarperHarper"`

When should we traverse dictionaries and lists? I think it's been mentioned before that traversing can be dangerous because it updates the list/dictionary as you go and can create unwanted situations (i.e. adding keys in a traversing for loop might also traverse over those new keys if that makes any sense)

The word "traverse"/"traversing" just means "loop over each element or pair one by one", which isn't dangerous at all! Things get more complicated if you start adding or removing elements from a dictionary or list while you're traversing over them, usually with a for loop. It's mostly "dangerous", or rather can cause problems, in the sense you may not get the result you want. The traversing itself doesn't update anything without you explicitly doing the update. In practice, we often create modified versions of the object (list or dictionary) by creating a copy and then updating that, so the original is intact for comparison and additional use, etc. It really all depends what you want the outcome of the code to be.